

Asynchronous Architecture in Tank Battle Game

Applying Asynchronous Programming in a Multi-player Tank Battle Game

Why Asynchronous Programming?

- Enables **concurrent, non-blocking execution** for handling player **inputs**, **bullet** movement, and **state** broadcasts simultaneously.
- Event-driven Architecture
 - **Responds** dynamically to player commands, movement ticks, and rendering loops.

System Modules (4 Core Components)

1. **Server** (server.py) - Central game authority managing rules, physics, and state.
2. **Bot** (bot.py) - Automated **AI player** sending decision commands.
3. **Tank** (tank.py) - **Human** player controller via non-blocking keyboard events.
4. **Dashboard** (web_dashboard.py) - Real-time visual display powered by WebSockets.

Server Module (server.py)
Asynchronous Event Loop

Server: The Heart of Game Rules Powered by asyncio

- Asyncio Event Loop
 - **Task 1:** Listen Commands
 - (Subscribe: Redis Pub/Sub)
 - **Task 2:** Game Loop (10 FPS)
 - (Move, Fire, Collide, Publish)

Asynchronous Implementation Details

- `asyncio.create_task()`: Runs two main tasks concurrently within a single-threaded Event Loop:
 - **Task 1: `listen_commands()`** — Non-blockingly listens for player actions (Move/Fire) from Redis Pub/Sub.
 - **Task 2: `game_loop()`** — Periodically updates game state physics (Tick Rate = 0.1s).
- **Non-blocking Sleep** Utilizes `await asyncio.sleep(TICK_RATE)` to yield control back to the Event Loop instead of blocking execution with `time.sleep()`.
- **State Broadcast**: Serializes and publishes the updated `game_state` via Redis Pub/Sub every **100ms**.

Bot Module (bot.py) — Non-blocking AI Execution

Bot: Independent Decision Making and Action Dispatch

- Each bot runs its own **isolated Event Loop**.
- Continuous Decision Loop
 - **Reads** the latest **game_state** asynchronously from **Redis** Pub/Sub.
 - **Calculates** optimal moves and **publishes** commands without waiting for other clients.
 - Controls action **frequency** using `await asyncio.sleep(delay)`.
- **Dozens** of bots can run concurrently without CPU overhead blocking or thread-locking issues.

Tank Module (tank.py) — Event-driven I/O Bound

Human Player: Non-blocking Keyboard Input Handling

- Keyboard Listener
 - **pynput Thread**
- Event Press
 - [Non-blocking Send]
 - (Redis Publish)

Asynchronous Implementation Details

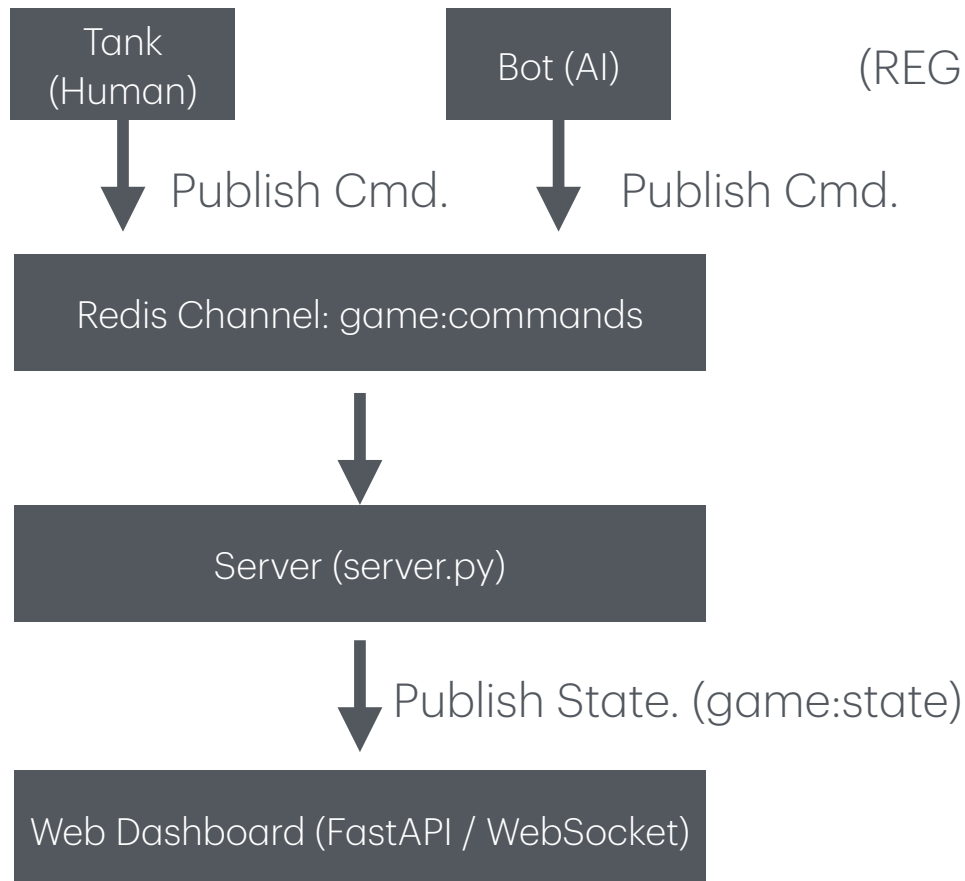
- **Non-blocking Keyboard Listener** Employs **pynput** on a background thread to capture keypresses (**Arrow Keys** / **Spacebar**) without freezing program execution (avoiding blocking `input()` calls).
- **Action Throttling & Cooldown** Enforces time-delta checks to limit command frequency and prevent input spam over the network.
- Asynchronous **State Subscription Continuously** listens for match termination events (Game Over / Winner announcement) while capturing user input simultaneously.

Dashboard Module
(web_dashboard.py) — Async Web &
WebSockets

Web Dashboard: Real-time Data Streaming with FastAPI & WebSockets

- **lifespan Task Execution** Spawns a background **redis_listener()** task to continuously **consume incoming state** updates from **Redis** Pub/Sub.
- **WebSocket Broadcasting** Asynchronously broadcasts state updates to all connected browser clients using **await** `connection.send_text()`.
- Client-side Rendering (HTML5 Canvas)
 - **Browser** client receives state updates asynchronously via **JavaScript ws.onmessage**.

Summary & Data Flow Architecture



(REGISTER, MOVE, FIRE)

```
{
  "grid_size": 10,
  "status": "WAITING",
  "players": {
    "Bot_1": {
      "x": 3,
      "y": 5,
      "hp": 100,
      "dir": "UP",
      "last_fire_time": 1718200000.0,
      "next_action": null
    }
  },
  "bullets": [
    {
      "x": 3,
      "y": 4,
      "dx": 0,
      "dy": -1,
      "owner": "Bot_1"
    }
  ],
  "winner": null,
  "time_left": 180,
  "countdown": 0
}
```

Key

- **Single Thread**, High Concurrency: Python's asyncio handles multiple concurrent I/O operations (Network & Redis Pub/Sub) with minimal resource consumption.
- **Decoupled Architecture**: Modules do not directly depend on or call each other; **communication** occurs strictly through **asynchronous message** passing over Redis.
- Responsive User Experience: UI rendering and gameplay mechanics remain smooth and unblocked regardless of the number of active players.